# SaveFits - Product Requirements Document

## Executive Summary

**Product Name:** SaveFits
**Mission:** Save outfit inspiration from anywhere, organize in collections, find with smart search
**Target:** Gen Z & Millennials (18-34) who save outfit inspiration across social platforms
**Timeline:** 2-month solo development MVP
**Tech Stack:** React Native + Expo, NestJS, PostgreSQL, Algolia

---

## 1. Product Overview

### 1.1 Problem Statement

Users save outfit inspiration across multiple platforms (TikTok, Instagram, Pinterest, Screenshots) but:

- Content is scattered and hard to find
- No unified organization system
- Poor search functionality
- Screenshots clutter camera roll

### 1.2 Solution

SaveFits provides a centralized platform to:

- Save outfits from any social platform via share extensions
- Auto-organize with AI-powered analysis
- Smart search across saved content
- Organize into custom collections

### 1.3 Success Metrics

- **User Acquisition:** 1,000 users in first month
- **Engagement:** Average 15 outfits saved per user
- **Retention:** 60% weekly active users
- **Processing:** <30 seconds outfit analysis time

---

## 2. User Personas & Use Cases

### 2.1 Primary Persona: Style Enthusiast Sarah

- **Age:** 22, College Student
- **Behavior:** Scrolls TikTok daily, saves 3-5 outfit inspirations weekly
- **Pain:** Screenshots everywhere, can't find saved looks when needed
- **Goal:** Quick save and easy retrieval of outfit inspiration

### 2.2 Core Use Cases

1. **Quick Save:** Share TikTok outfit → SaveFits processes → Back to TikTok (15 seconds)
2. **Organization:** Group outfits into "Work," "Date Night," "Casual" collections
3. **Discovery:** Search "red dress casual" → Find relevant saved outfits
4. **Inspiration:** Browse collection before shopping/getting dressed

---

## 3. Functional Requirements

### 3.1 Core Features (MVP)

#### 3.1.1 Content Saving

**FR-001: Social Media Integration**

- **Description:** Save outfits from TikTok, Instagram, Pinterest via share
- **Acceptance Criteria:**
  - Share extension appears in social apps
  - Content processes in background (≤30 seconds)
  - Status indicators (Processing, Ready, Failed)
  - Original source link preserved

**FR-002: Camera Roll Import**

- **Description:** Save outfits from existing photos
- **Acceptance Criteria:**
  - Photo picker integration
  - Batch upload support
  - Auto-detect recent screenshots
  - Manual override for "no outfit detected"

### 3.1.2 Content Organization

#### FR-003: Collections System

- **Description:** Organize outfits into custom collections
- **Acceptance Criteria:**
  - Create/edit/delete collections
  - Add outfits to multiple collections
  - Collection sharing via links
  - Visual collection previews

#### FR-004: Tagging System

- **Description:** Auto and manual tagging for better discovery
- **Acceptance Criteria:**
  - AI-generated tags (colors, styles, clothing types)
  - User-added custom tags
  - Tag-based filtering
  - Tag editing and management

### 3.1.3 Content Discovery

#### FR-005: Smart Search

- **Description:** Find saved outfits with natural language
- **Acceptance Criteria:**
  - Full-text search across tags, descriptions
  - Visual similarity search
  - Filter by collections, dates, sources
  - "Search online" fallback option

#### FR-006: Content Feed

- **Description:** Browse all saved outfits in organized grid
- **Acceptance Criteria:**
  - Infinite scroll grid layout
  - Quick view with tap-to-expand
  - Sort by date, popularity, collections

- Pull-to-refresh updates

#### 3.1.4 AI Processing

**FR-007: Outfit Analysis**

- **Description:** Automated outfit detection and classification
- **Acceptance Criteria:**
  - Binary outfit detection (yes/no)
  - Color extraction and tagging
  - Style classification (casual, formal, etc.)
  - Confidence scoring

### 3.2 Enhanced Features (Phase 2)

- Individual clothing item extraction
- Outfit recommendations based on saved content
- Social sharing and community features
- Wishlist functionality
- Advanced analytics (SaveFits Wrapped)

---

## 4. Technical Architecture

### 4.1 Technology Stack

#### 4.1.1 Frontend (Mobile App)

- **Framework:** React Native 0.73+ with Expo SDK 50+
- **State Management:** Redux Toolkit + RTK Query
- **Navigation:** React Navigation v6
- **UI Components:** NativeBase or React Native Elements
- **Image Handling:** React Native Fast Image
- **Share Extension:** Custom iOS/Android share extensions

#### 4.1.2 Backend (API)

- **Framework:** NestJS with TypeScript
- **Database:** PostgreSQL 15+ with Prisma ORM

- **Authentication:** JWT with Passport.js
- **File Storage:** AWS S3 with CloudFront CDN
- **Background Jobs:** Bull Queue with Redis
- **API Documentation:** Swagger/OpenAPI

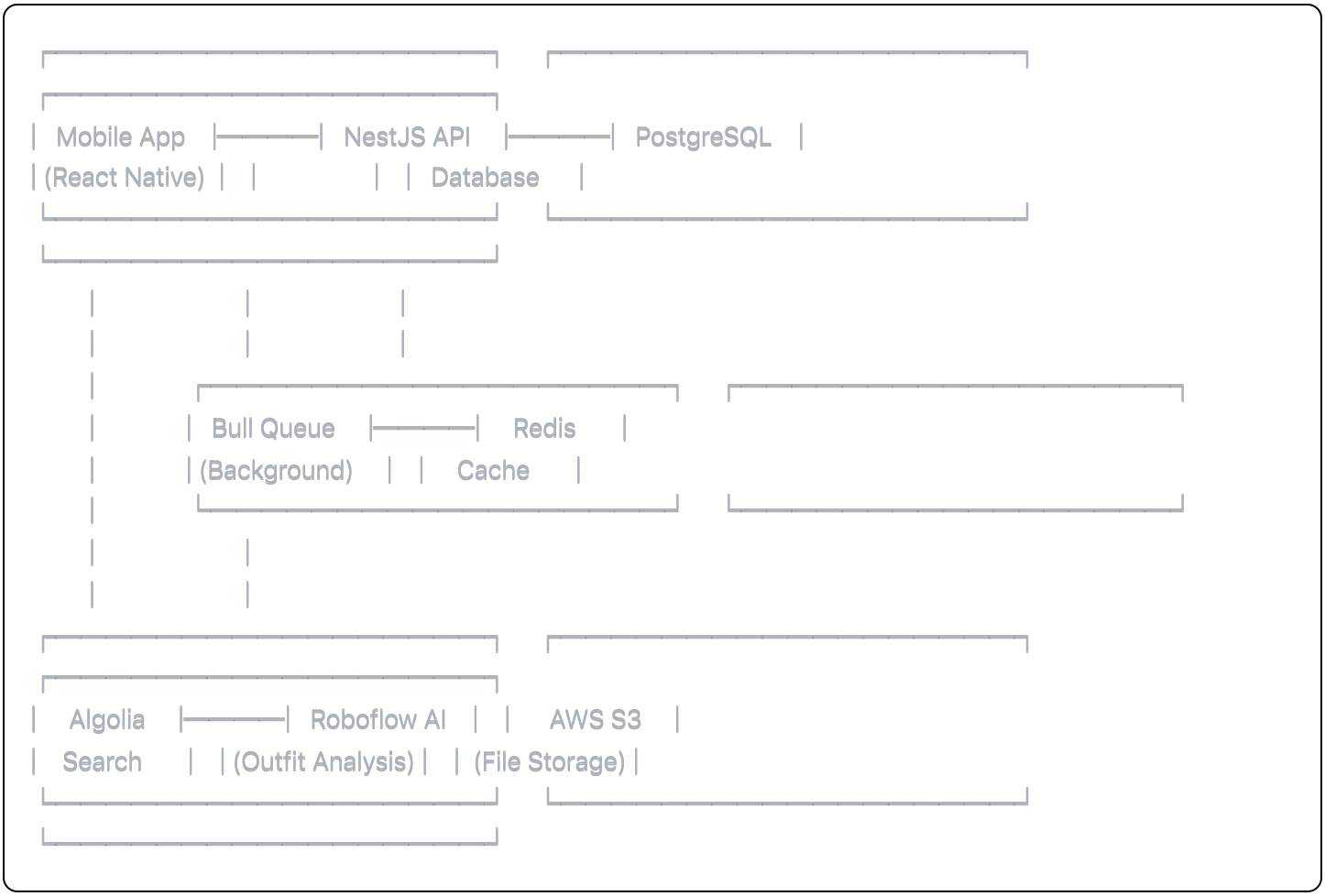
#### 4.1.3 AI & Search

- **Computer Vision:** Roboflow Clothing Detection API
- **Search Engine:** Algolia (managed search service)
- **Image Processing:** Sharp.js for thumbnails/optimization
- **Video Processing:** FFmpeg for frame extraction

#### 4.1.4 Infrastructure

- **Hosting:** AWS ECS Fargate (backend), Expo Application Services (mobile)
- **Database:** AWS RDS PostgreSQL
- **Cache:** AWS ElastiCache (Redis)
- **Monitoring:** Sentry for error tracking
- **Analytics:** Mixpanel for user behavior

### 4.2 System Architecture




## 4.3 Database Schema

sql

```
-- Users
users {
  id: UUID PRIMARY KEY
  email: VARCHAR UNIQUE
  username: VARCHAR UNIQUE
  first_name: VARCHAR
  last_name: VARCHAR
  profile_image_url: VARCHAR
  created_at: TIMESTAMP
  updated_at: TIMESTAMP
}

-- Outfits
outfits {
  id: UUID PRIMARY KEY
  user_id: UUID FOREIGN KEY
  status: ENUM('processing', 'ready', 'failed', 'no_outfit')
  source_url: VARCHAR
  source_type: ENUM('tiktok', 'instagram', 'pinterest', 'screenshot', 'photo')
  image_url: VARCHAR
  thumbnail_url: VARCHAR
  original_text: TEXT
  ai_tags: JSONB
  user_tags: VARCHAR[]
  colors: VARCHAR[]
  style_category: VARCHAR
  confidence_score: FLOAT
  analysis_data: JSONB
  created_at: TIMESTAMP
  updated_at: TIMESTAMP
}

-- Collections
collections {
  id: UUID PRIMARY KEY
  user_id: UUID FOREIGN KEY
  name: VARCHAR
  description: TEXT
  is_public: BOOLEAN DEFAULT false
  thumbnail_url: VARCHAR
  created_at: TIMESTAMP
  updated_at: TIMESTAMP
}
```

```
-- Outfit Collections (Many-to-Many)
outfit_collections {
  outfit_id: UUID FOREIGN KEY
  collection_id: UUID FOREIGN KEY
  added_at: TIMESTAMP
  PRIMARY KEY (outfit_id, collection_id)
}

-- User Analytics
user_analytics {
  id: UUID PRIMARY KEY
  user_id: UUID FOREIGN KEY
  total_outfits: INTEGER
  dominant_colors: VARCHAR[]
  style_breakdown: JSONB
  monthly_saves: JSONB
  streak_data: JSONB
  created_at: TIMESTAMP
  updated_at: TIMESTAMP
}
```

---

# 5. API Specifications

## 5.1 Core Endpoints

### Authentication

typescript

```
POST /auth/register
POST /auth/login
POST /auth/refresh
POST /auth/logout
```

### Outfits

typescript

```
POST /outfits                 // Create new outfit
GET /outfits                  // List user's outfits
GET /outfits/:id              // Get specific outfit
PUT /outfits/:id              // Update outfit
DELETE /outfits/:id           // Delete outfit
POST /outfits/:id/analyze     // Trigger re-analysis
```

### Collections

typescript

```
POST /collections             // Create collection
GET /collections              // List user's collections
GET /collections/:id          // Get collection details
PUT /collections/:id          // Update collection
DELETE /collections/:id       // Delete collection
POST /collections/:id/outfits   // Add outfit to collection
DELETE /collections/:id/outfits/:outfitId // Remove from collection
```

### Search

typescript

```
GET /search/outfits?q={query}&filters={filters}
GET /search/suggestions?q={partial_query}
```

## 5.2 Webhook Endpoints

typescript

```
POST /webhooks/roboflow       // Receive analysis results
POST /webhooks/upload-complete  // File upload completion
```

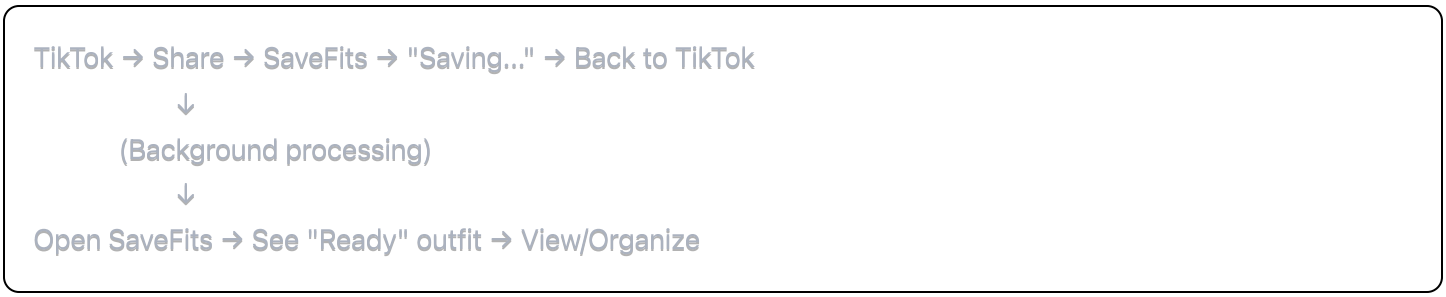
---

# 6. User Experience Design

## 6.1 App Navigation Structure


### 6.2 Key User Flows

#### Primary Flow: Save from TikTok




#### Secondary Flow: Browse & Search


### 6.3 Share Extension Design

- **iOS:** Native iOS Share Extension with custom UI
- **Android:** Intent filter with processing activity
- **Processing Time:** Maximum 15 seconds in foreground

---

## 7. Implementation Plan (2 Months)

### Week 1-2: Foundation

- [ ] Set up development environment (Expo, NestJS)
- [ ] Basic authentication system
- [ ] Database schema and migrations
- [ ] File upload to S3
- [ ] Basic mobile app navigation

### Week 3-4: Core Features

- [ ] Outfit saving from camera roll
- [ ] Basic AI analysis integration (Roboflow)
- [ ] Collections CRUD operations
- [ ] Search implementation (Algolia)
- [ ] Background job processing

### Week 5-6: Share Extensions

- [ ] iOS share extension development
- [ ] Android intent handling
- [ ] Social media integration testing
- [ ] Error handling and retry logic

### Week 7-8: Polish & Launch

- [ ] UI/UX refinements
- [ ] Performance optimization
- [ ] Testing (unit, integration, E2E)
- [ ] App store submission preparation
- [ ] Beta testing with friends

## 8. Risk Assessment & Mitigation

#### High-Risk Items

1. **Share Extension Complexity**
   - **Risk:** Platform-specific implementation challenges
   - **Mitigation:** Start with one platform, iterate quickly
2. **AI Analysis Accuracy**
   - **Risk:** Poor outfit detection results
   - **Mitigation:** "Save anyway" option, manual tagging
3. **Performance at Scale**
   - **Risk:** Slow loading with many outfits
   - **Mitigation:** Pagination, image optimization, caching

#### Medium-Risk Items

1. **User Adoption**
   - **Risk:** Low initial engagement
   - **Mitigation:** Focus on core value prop, gather feedback

2. **Content Moderation**
    - **Risk:** Inappropriate content
    - **Mitigation:** Start with private saves only

---

## 9. Success Metrics & KPIs

### 9.1 User Metrics

- **DAU/MAU Ratio:** Target 25%+ (daily/monthly active users)
- **Retention:** 60% Week 1, 30% Month 1
- **Saves per User:** Average 15 outfits per user
- **Search Usage:** 40% of users search weekly

### 9.2 Technical Metrics

- **API Response Time:** <200ms for 95th percentile
- **Analysis Processing:** <30 seconds average
- **App Crash Rate:** <1% of sessions
- **Search Relevance:** >80% user satisfaction

### 9.3 Business Metrics

- **User Acquisition Cost:** Track via attribution
- **Monthly Active Users:** Target 1,000 in Month 1
- **Feature Adoption:** 70% use collections, 50% use search

---

## 10. Post-MVP Roadmap

### Month 3-4: Enhanced Features

- Individual clothing item detection
- Outfit recommendations engine
- Social sharing improvements
- Wishlist functionality

### Month 5-6: Community Features

- Public collections
- User following

- Outfit rating system
- Community challenges

### Month 7-8: Monetization

- Premium features (unlimited saves, advanced search)
- Shopping partnerships and affiliate links
- SaveFits Wrapped annual feature

---

## 11. Appendices

### A. Technology Alternatives Considered

| Feature | Chosen | Alternative | Reason |
|---|---|---|---|
| Mobile Framework | React Native + Expo | Flutter | Faster development, web sharing |
| Backend | NestJS | Express | Better structure, TypeScript |
| Database | PostgreSQL | MongoDB | ACID compliance, relations |
| Search | Algolia | Elasticsearch | Managed service, faster setup |
| AI/ML | Roboflow | Custom ML | Pre-trained, faster to market |

### B. Cost Estimates (Monthly)

- **Hosting (AWS):** $100-200
- **Roboflow API:** $50-150
- **Algolia:** $50-100
- **File Storage:** $25-50
- **Total:** ~$225-500/month

### C. Legal Considerations

- Terms of Service for user-generated content
- Privacy Policy for data collection
- DMCA compliance for shared content
- Age restrictions (13+ with parental consent)

---

**Document Version:** 1.0
**Last Updated:** January 2025
**Status:** Ready for Development